Chapter 3

The µML Task Model

Contents

3. The µML Task Model

The *µML Task Model* is constructed during requirements capture and is one of the models that must be provided from the start. When sketched by hand, a Task Model may be constructed incrementally, which allows the engineer to identify tasks at different levels of granularity. When formally defined, a Task Model provides a complete, structured model of the business activity.

The purpose of the Task Model is to identify clusters of business tasks and link these to the human stakeholders who perform them. The Task Model can be cross-checked against an Impact Model, and can be used to derive a State Model of the intended system's user interface.

3.1 Elements of the Task Model

The Task Model consists of the following concepts:

- **Task** – is a unit of business activity. A task may be *atomic*, representing a simple business activity[1], or *composite*, consisting of other tasks[2]. A task is always concrete, achieving a measurable business objective.
- **Goal** – is an abstract kind of task, which has an aim that cannot be measured directly. A goal is a generalisation of several tasks having the same aim. A task may realise a goal, achieving an objective which satisfies the aim.
- **Actor** – is a stakeholder, who is involved in the business. An actor is any external entity who interacts with the business. An actor can be an *operator*, if they are in control of a task, or a *beneficiary*, if they are otherwise affected by a task.
- **System** – is a kind of actor, representing an external computer system, rather than a human stakeholder, that interacts with the business.

The Task Model also consists of the following connectives:

- **Association** – linking an actor to a task involving that actor. The association may declare that the actor enacts the task (viz. is the operator), otherwise the actor is a beneficiary.
- **Generalisation** – linking any actor to a more general actor; or linking any task to a more general task, or linking any task to a goal. This expresses the "a kind of" relationship. The source is more specific, and the target is more general.
- **Composition** – linking any task to a composite task. This expresses the "a part of" relationship. The source is a component part, and the target is the composite whole. Composition is transitive, so composite tasks may consist of atomic and composite tasks.

The Task Model also has the following regions:

- **Boundary** – is a kind of region enclosing the tasks that will be provided as part of a system, or as part of a specific version or release of the system, excluding those tasks which will not be part of this. A boundary denoting a later release should enclose any earlier release.
- **Diagram** – is an implicit kind of region enclosing everything in the Task Model. The diagram encloses any boundary and also any tasks that were outside the boundary. If no boundary is given, the entire diagram is assumed to be the intended boundary.

3.2 Notation for the Task Model

Figure 3.1 illustrates the notation used for the Task Model. It adopts standard µML conventions in the concepts and connectives used. In particular, elements with dashed outlines represent more abstract

[1] An atomic task corresponds to a use case in UML.
[2] A composite task has no correspondence in UML.

versions of the same things with solid outlines; and the relationships expressed by the connectives have the same meanings as in other µML models.

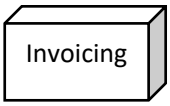
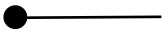
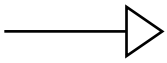
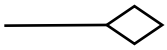
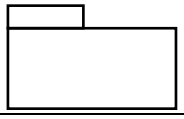
Task: denotes a business activity with a measurable, concrete objective. Drawn as an ellipse with a solid outline, containing the name of the task.	
Goal: denotes an abstract task, specifying general, abstract aim, which cannot be measured. Drawn as an ellipse with a dashed outline, containing the name of the goal.	
Actor: denotes a human entity, a stakeholder interacting in a particular role, with one or more tasks. Drawn as a stick figure with a label positioned below containing the name.	
System: denotes a machine entity, typically a computer system or subsystem interacting with one or more tasks. Drawn as a named 3D box with side shading.	
Association: denotes the "related to" relationship between two different concepts. This relationship is not directed. Drawn as a straight line connecting an actor with a task.	
Enacts adornment: drawn as a small black filled circle at the end of the association next to the actor. Denotes ownership by the actor, who enacts the related task.	
Generalisation: denotes the "a kind of" relationship between a specific and a general concept. The triangular arrowhead points to the general concept.	
Composition: denotes the "a part of" relationship between a part and a whole concept. The diamond arrowhead points to the whole concept.	
Boundary: denotes the scope of the system, or of a version or release of the system. Drawn as an enclosing rectangle with a tab, in which the system or version name is written.	

Figure 3.1: Notation for the Task Model

3.3 Rules of the Task Model

The Task Model has the following syntactic and semantic rules:

- **Atomic task** – an atomic task must be one which creates, deletes, updates, deletes, sends or receives some single piece of information (a definition rule).
- **Composite task** – a composite task should consist of at least two or more other tasks, which may be atomic or composite tasks (a definition rule).
- **Task clustering** – every atomic task must belong to a cluster, that is, every atomic task must eventually belong to some composite task (a completeness rule).
- **No ordering** – there is no implied order among the clustered tasks of a composite task, nor is there any implied order among the descendants of a general task (a semantic rule).

- **Logical and-tree** – a task tree connected by composition may be interpreted as a logical and-tree. To execute the composite task means executing each of its component tasks (a semantic rule).
- **Logical or-tree** – a task tree connected by generalisation may be interpreted as a logical or-tree. To execute the general task means executing one or other of the specific tasks, which are disjoint (a semantic rule).
- **General flattening** – any relationship targeting a general node is considered equivalent to a set of relationships targeting all the specific nodes under it individually (an elimination rule).
- **Composite flattening** – any relationship targeting a composite node is considered equivalent to a set of relationships targeting all the component nodes under it individually (an elimination rule).
- **Implicit boundary** – any Task Model with no boundary is interpreted as though the enclosing diagram is the boundary. Every contained task is then part of the system (a completeness rule).

A consequence of these rules is that a fully connected Task Model may be interpreted as an AND-OR tree, which offers other formal possibilities, such as logical rules of distribution.

3.4 Task Model Examples

Some examples are given below of Task Models. The first example describes a Lending Library. The second example describes a Training Agency.

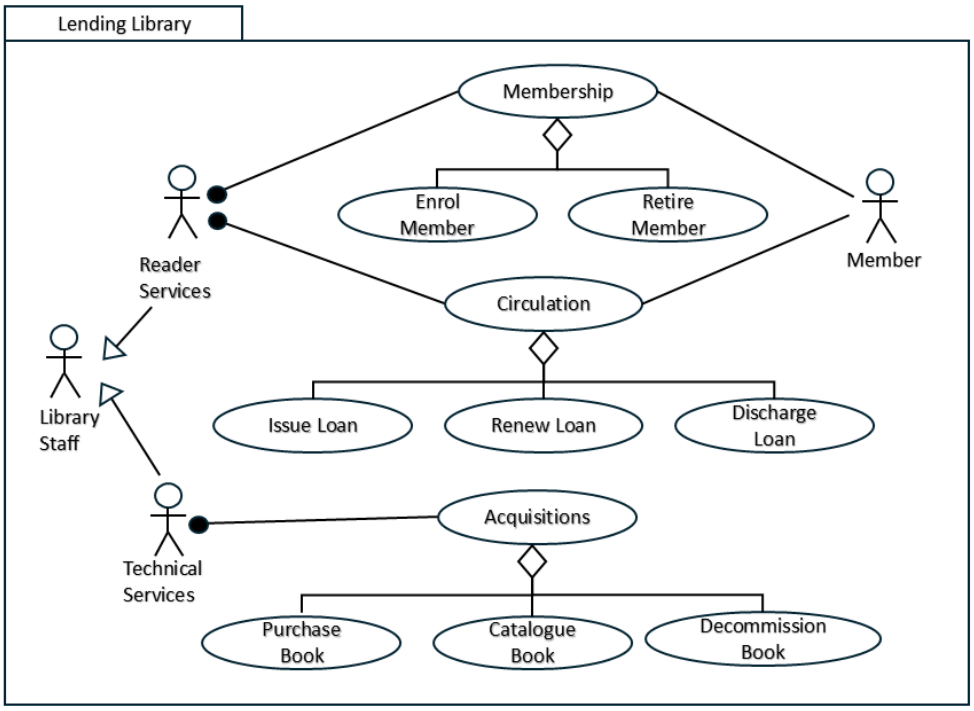


Figure 3.2: Task Model for a Lending Library

Figure 3.2 illustrates an example Task Model for a Lending Library, which issues loans of books to its members. The model consists of eight atomic tasks, clustered under three composite tasks:

- **Membership** – dealing with library membership, consists of Enrol Member and Retire Member.
- Circulation – dealing with the day-to-day lending of books, consists of Issue Loan, Renew Loan and Discharge Loan.
- **Acquisitions** – dealing with the management of books, consists of Purchase Book, Catalogue Book and Decommission Book.

There are three main actors: Member, Reader Services and Technical Services. The latter two are also identified as kinds of Library Staff, by generalisation.

- **Reader Services** – operates all Circulation and Membership tasks.
- **Technical Services** – operates all Acquisitions-related tasks.
- **Member** – is involved in all Circulation and Membership tasks as a beneficiary.

All the specified tasks lie within the boundary of the Lending Library. The positioning of the actors is irrelevant, since all actors are external to the system by definition.

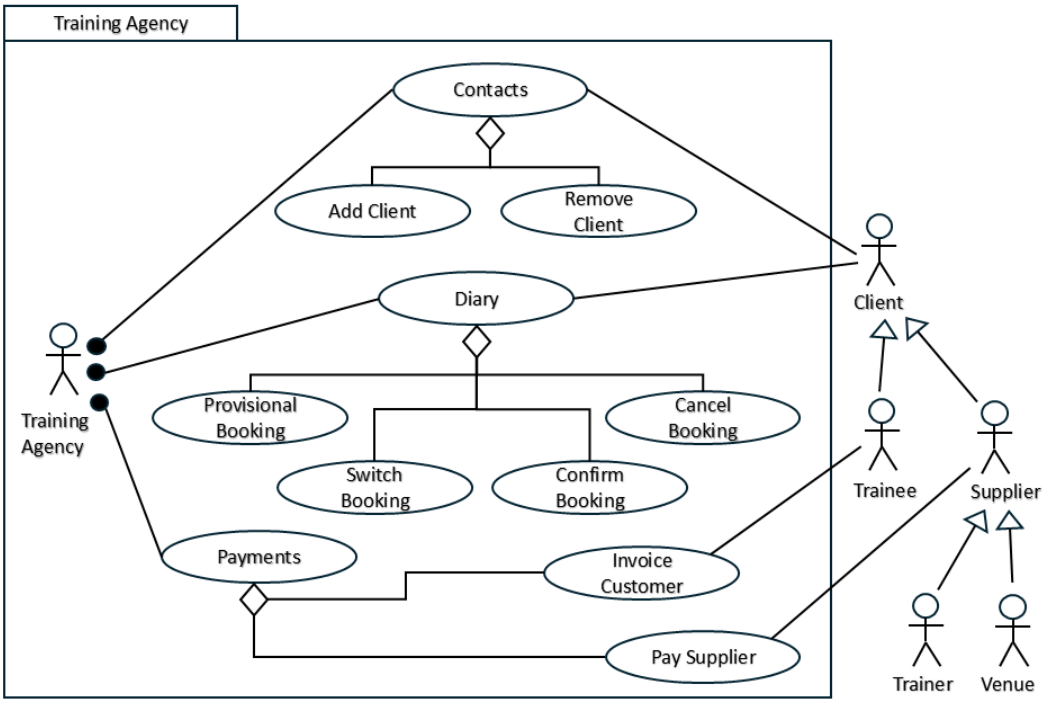


Figure 3.3: Task Model for a Training Agency

Figure 3.3 illustrates a Task Model for a Training Agency, which arranges training sessions for trainees with trainers, booked into various venues. The model consists of eight atomic tasks clustered under three composite tasks:

- **Contacts** – dealing with the client list of the agency, consists of Add Client and Remove Client. Note that there are three concrete kinds of client: Trainee, Trainer and Venue.
- **Diary** – dealing with the day-to-day management of bookings, consists of Provisional Booking, Switch Booking, Confirm Booking and Cancel Booking. These involve all kinds of Client.
- **Payments** – dealing with the financial aspects, consists of Invoice Customer and Pay Supplier. Only the Trainee is invoiced, but the paid Suppliers include the Trainer and Venue.

There are four main actors, some of whom are generalised as Supplier, or Client:

- **Training Agency** – is the operator of the agency, which is shown using the enacts-adornment on all associations.
- **Trainee** – is a kind of Client, who hires the agency to organise some training, benefits from the training and pays for it.
- **Trainer** – is a kind of Supplier, who offers their training services to the agency and is paid for this.
- **Venue** – is a kind of Supplier, who provides a conference room for the training session and is paid for this.

This example involves a greater use of generalisation over the different groups of clients. One thing which is not explicit is how many clients of which kinds are involved in any single task. For example, adding a client will add either a Trainee, a Trainer or a Venue. But making a provisional booking will involve all three of a Trainee, a Trainer and a Venue. This level of detail must be obtained from a different model.

3.5 Task Metamodel

The elements of the µML Task Model may be defined in a metamodel. In figure 3.4, we use a metamodel written in the *ReMoDeL* metamodel definition language [1]. This extends and redefines concepts defined in the ReMoDeL metamodel of the µML Core Model, given in Chapter 2.

```
# The Task Model from muML (Micro Modelling Language).  This consists of
# Tasks associated with Actors, of which one enacts the Task.  Tasks may
# be grouped by composition or generalisation.  Actors may be grouped by
# generalisation.

metamodel TaskModel inherit CoreModel {

# Classifier is a kind of Concept that enters into generalisation
# and composition relationships.  A Classifier belongs to a Diagram.

   concept Classifier inherit Concept {
      reference owner : Diagram
      operation isAbstract : Boolean {
         owner.generalisations.exists(gen | gen.general = self)
      }
      operation isConcrete : Boolean {
         not self.isAbstract
      }
      operation isComposite : Boolean {
         owner.compositions.exists(comp | comp.whole = self)
      }
      operation isComponent : Boolean {
         owner.compositions.exists(comp | comp.part = self)
      }
      operation isAtomic : Boolean {
         not (self.isAbstract or self.isComposite)
      }
   }

# Actor represents an external agency that interacts with a Task.  An
# Actor is typically a human agent; but a subtype of Actor is System,
# an external system.

   concept Actor inherit Classifier {
      operation isSystem : Boolean { false }
      operation supertypes : Actor[] {
         owner.actors.select(act | owner.generalisations.exists(gen |
            gen.specific = self and gen.general = act))
      }
      operation subtypes : Actor[] {
         owner.actors.select(act | owner.generalisations.exists(gen |
            gen.specific = act and gen.general = self))
      }
      operation isOperator(task : Task) : Boolean {
         owner.associations.exists(assoc | assoc.enacts and
            assoc.source = self and assoc.target = task)
         or task.composites.exists(comp | self.isOperator(comp))
         or task.supertypes.exists(sup | self.isOperator(sup))
      }
   }

# System is a kind of Actor representing an external system that
# interacts with a Task.
```

```
   concept System inherit Actor {
      operation isSystem : Boolean { true }
   }

# Task represents a business activity, having a concrete objective.  A
# Task may be a composition or generalisation of other Tasks, or an atomic
# Task.  The atomic Tasks must match those found in the Impact Model.

   concept Task inherit Classifier {
      operation supertypes : Task[] {
         owner.tasks.select(task | owner.generalisations.exists(gen |
            gen.specific = self and gen.general = task))
      }
      operation subtypes : Task[] {
         owner.tasks.select(task | owner.generalisations.exists(gen |
            gen.specific = self and gen.general = task))
      }
      operation components : Task[] {
         owner.tasks.select(task | owner.compositions.exists(comp |
            comp.part = task and comp.whole = self))
      }
      operation composites : Task[] {
         owner.tasks.select(task | owner.compositions.exists(comp |
            comp.part = self and comp.whole = task))
      }
      operation actors : Actor[] {
         owner.associations.select(assoc | assoc.target = self)
            .collect(assoc | assoc.actor).asSet
         .union(self.composites.collate(task | task.actors))
         .union(self.supertypes.collate(task | task.actors)).asList
      }
   }

# Goal is a kind of Task denoting an abstract Goal.  A Goal has no
# concrete objective, but has an abstract aim.  A Goal may generalise
# over several Tasks that satisfy the goal.

   concept Goal inherit Task {
      operation isAbstract : Boolean { true }
   }

# Relationship is a kind of Connective relating two Classifiers.
# A Relationship belongs to a Diagram.

   concept Relationship inherit Connective {
      reference owner : Diagram
      reference source : Classifier
      reference target : Classifier
   }

# Association represents a directed Relationship between a source
# Actor and a target Task.  If enacts is true, this indicates that
# the Actor is the operator of the Task.

   concept Association inherit Relationship {
      attribute enacts : Boolean
      operation actor : Actor { source }
      operation task : Task { target }
   }

# Composition represents a directed parts-wholes relationship between two
# Concepts.  The source is the part-Concept and the target is the
# whole-Concept.

   concept Composition inherit Relationship {
     operation part : Task { source }
     operation whole : Task { target }
   }
```

```
# Generalisation represents a directed a-kind-of relationship between two
# Concepts.  The source is the specific-Concept and the target is the
# general-Concept.  Generalisation is used among both Actors and Tasks
# (linking a Task to a Goal).

   concept Generalisation inherit Relationship {
     operation specific : Classifier { source }
     operation general : Classifier { target }
   }

# Boundary represents a partition of the Task Model, including some Tasks
# but excluding others.  When present in a Diagram, any Tasks outside the
# top-level Boundary will not be implemented in the system.  If a Boundary
# contains a nested Boundary (for an earlier release), it also includes
# the Tasks of that release.

   concept Boundary inherit Region {
      reference owner : Diagram
      reference tasks : Task[]
      component nested : Boundary
      operation includes(elem : Task) : Boolean {
         tasks.exists(task | task = elem) or
           (nested /= null and nested.includes(elem))
      }
      operation excludes(elem : Task) : Boolean {
         not self.includes(elem)
      }
   }

# The Task Diagram is the root of the Task Model, containing all other
# components.  Apart from actors and tasks, the associations relate an
# Actor to a Task, the compositions relate a part-Task to a whole-Task,
# and the generalisations either relate a specific Actor to a general
# Actor, or a specific Task to a general Task.  The Boundary, if present,
# refers to Tasks that are included in the implemented system.

   concept Diagram inherit Region {
      component actors : Actor[]
      component tasks : Task[]
      component associations : Association[]
      component compositions : Composition[]
      component generalisations : Generalisation[]
      component boundary : Boundary
      operation inTasks : Task[] {
         tasks.select(task | boundary == null
            or boundary.includes(task))
      }
      operation outTasks : Task[] {
         tasks.select(task | boundary /= null
            and boundary.excludes(task))
      }
   }
}
```

Figure 3.4: Metamodel of the Task Model in the ReMoDeL language.

This provides the necessary elements of the Task Model via a number of intermediate abstractions in the metamodel hierarchy:

- **Classifier** – is a kind of Concept generalising over Actor and Task, which enters into generalisation and composition relationships;
- **Actor** – is a kind of Classifier, which enters into generalisation and association relationships;
- **System** – is a specialised kind of Actor;
- **Task** – is a kind of Classifier, which enters into generalisation, composition and association relationships;
- **Goal** – is an abstract kind of Task;

- **Relationship** – is a kind of Connective linking a source and target Classifier;
- **Association** – is a kind of Relationship linking an Actor and a Task;
- **Composition** – is a kind of Relationship linking a part and a whole Task;
- **Generalisation** – is a kind of Relationship linking a specific and general Classifier;
- **Boundary** – is a kind of Region enclosing Tasks and optionally a nested Boundary;
- **Diagram** – is a kind of Region that contains all the elements of the model.

3.6 Model Transformations

The µML Task Model is a source or target in a number of model transformations. Some of these are for the sake of consistency checking across different views, and some are to develop more concrete views of the software system.

- **ImpactToTaskModel** – is a mapping transformation that takes a µML Impact Model as the source, and from this constructs a default target µML Task Model, when no other Task Model exists. The result clusters atomic tasks according to whether they manipulate the same objects and defines a single default user as the only actor. No boundary is introduced.
- **MergeImpactWithTask** – is a merging transformation that takes an existing µML Task Model and a µML Impact Model as the sources, and from this constructs an updated target µML Task Model, which is consistent with extra information supplied in the Impact Model. The result attaches new atomic tasks to suitable clusters in the original Task Model, using the original actors. Any existing boundary will include relevant new tasks.
- **TaskToStateModel** – is a mapping transformation that takes a µML Task Model as the source, and from this constructs a µML State Model. The result is a finite state machine of the user interface for the business application represented by the original Task Model. Only tasks implemented in the system are considered. Composite tasks are mapped to states, and atomic tasks are mapped to transitions in those states. Additional transitions are created to map between a home state and the other derived states.

These transformations are too large to display here.

3.7 Differences from UML

The µML Task Model is most directly comparable with the UML Use-Case Diagram [2, 3, 4]. The biggest difference is that the *Task* concept stands for something of variable granularity, whereas a *Use-Case* has a specific atomic granularity [3, 4]. *Tasks* can be atomic, or composite consisting of other *Tasks*. Other pre-UML design methods captured *Tasks* in this way, including SOMA [5], OPEN [6, 7, 8] and the Discovery Method [9].

This is mostly intended to provide conceptual support to the requirements engineer, who is unlikely to elicit *Tasks* of uniform granularity in early interviews, therefore needs a notation that supports composition and decomposition, to help arrange Tasks by granularity. However, there is an equally important formal reason for providing a compositional algebra of Tasks [10], since this supports model transformations that use distributive and elimination rules (see section 3.3). The clustering of *Tasks* by Actor-involvement is also useful in developing GUIs for particular Actor roles.

The other main difference is that µML *Tasks* are linked structurally using the standard *Generalisation* and *Composition* connectives, whereas UML offered different *include* and *extend* dependencies, which do not have the same semantics and are frequently misused by engineers [11]. The *include* dependency relates a *Use-Case* to a shared sub-sequence of events. The *extend* dependency relates an optional subsequence of events to a main *Use-Case*. The sub-sequences are smaller than a Use-Case in granularity and should not be treated as *Use-Cases*.

Furthermore, the UML *extend* dependency is directed in the reverse order of true logical dependency of a whole on its parts. The reason for this anomaly is that UML confuses *logical dependency* and *document dependency.* In Jacobson's original writings, it is clear that the reason for the dependency relationships

was to offer an economy in the way *Use-Cases* were documented. The source documents depended on the existence of prior target documents, for clear understanding.

Unfortunately, engineers have wrongly tended to promote these two UML dependencies into control structures corresponding to *go-to* and indeed *come-from* statements, in the worst tradition of pre-block-structured programming [11]. This is despite the original UML authors' recommendation that engineers should not forwards-engineer a *Use-Case* Diagram [3], p239. We have provided instead the µML Task Model, which is more amenable to forwards-engineering.

3.8 References

[1] AJH Simons, ReMoDeL Explained (rev 2.2): an introduction to ReMoDeL by example. *Technical Report, 25 January*, University of Sheffield (2023). https://staffwww.dcs.shef.ac.uk/people/A.Simons/remodel/current/ReMoDeL_Explained_25Jan23.pdf

[2] I Jacobson, M Ericsson and A Jacobson. *The Object Advantage: Business Process Re-engineering with Object Technology*, Addison-Wesley (1994)

[3] G Booch, J Rumbaugh, I Jacobson. *The Unified Modeling Language User Guide*, Addison-Wesley (1999).

[4] A Cockburn. *Writing Effective Use Cases*, Addison-Wesley (2001).

[5] I Graham. *Migrating to Object Technology*, Addison-Wesley (1995).

[6] DG Firesmith, B Henderson-Sellers and I Graham. *The OPEN Modeling Language (OML) Reference Manual*, SIGS Books, Cambridge University Press (1997).

[7] I Graham, B Henderson-Sellers and H Younessi. *The OPEN Process Specification*, Addison-Wesley (1997).

[8] B Henderson-Sellers, AJH Simons and H Younessi. *The OPEN Toolbox of Techniques*, Addison-Wesley (1998).

[9] AJH Simons. Object Discovery: a Process for Developing Medium-Sized Applications. *Tutorial 14, ECOOP 1998 Tutorials*, AITO/ACM (1998). http://staffwww.dcs.shef.ac.uk/people/A.Simons/research/tutorials/ecoop98tut14.pdf

[10] CA Fernández-y-Fernández and AJH Simons. An algebra to represent task flow models. *International Journal of Computational Intelligence Theory and Practice*, **6** (2), Serials Publications (2011), 63-74.

[11] AJH Simons. Use cases considered harmful. In: *Technology of Object-Oriented Languages and Systems*, **29**, eds. R. Mitchell, A. C. Wills, J. Bosch, B. Meyer, IEEE Computer Society (1999), 194-203. https://eprints.whiterose.ac.uk/200754/